

EXAMPLE PRODUCT · FRONTEND ENGINEER

# 업무용 웹 제품의 상태 흐름과 입력 경험을 개선합니다

React와 TypeScript로 운영자가 반복하는 확인, 입력, 리포트 흐름을 줄이는 UI 구조를 설계하고 구현합니다.

## FOCUS 01

### Dashboard

서버 상태와 URL 필터 상태를 분리해 공유 가능한 운영 화면 구성

## FOCUS 02

### Validation

Zod 스키마와 타입 안정성으로 QA 재현 이슈를 입력 단계에서 차단

# Sample Dashboard

운영자가 여러 CSV를 합쳐 보던 주간 지표 확인 흐름을 한 화면의 대시보드로 정리했습니다.

## Problem

필터 조건을 공유하기 어렵고, 같은 조건으로 다시 확인하는 과정이 반복적이었습니다.

## Decision

서버 데이터 상태와 사용자 필터 상태를 분리하고, 필터 조건은 URL query에 동기화했습니다.

## Action

TanStack Query로 로딩, 재요청, 서버 상태를 관리하고 Recharts로 주요 지표와 추세를 시각화했습니다.

## Result

운영자로부터 주간 리포트 작성 시간이 2시간에서 30분으로 줄었다는 피드백을 받았습니다.

## JD 연결점

대시보드 UI, API 연동 구조, 서버 상태 관리, 운영자 흐름 개선 요구와 직접 연결됩니다.

# Sample Company Form Validation

QA 단계에서 반복 발견되던 폼 데이터 오류를 입력 단계에서 차단하도록 스키마와 타입을 정리했습니다.

## 01 Problem

폼 데이터 타입이 느슨해 런타임 오류가 QA 단계에서 반복 발견됐습니다.

## 02 Decision

입력 단계에서 오류를 차단하도록 폼 스키마와 API 응답 타입을 명시했습니다.

## 03 Action

Zod 스키마, 명시 타입, 검증 실패 메시지 매핑을 추가했습니다.

## 04 Result

QA 재현 이슈 5건 중 4건을 사전 검증 단계에서 차단했습니다.

### 실제 제출 전 교체할 증빙

GitHub repository, PR 링크, 화면 스크린샷, 사용자 피드백 원문, before/after 측정 방식